[CS3704] Intermediate Software Design and Engineering

Dr. Chris Brown
Virginia Tech
10/20/2026

Announcements

- **PM2.1 [Development Sprint I] due Friday (10/23) before midnight**
- **HW3 due next Friday (10/30) before midnight**
  - **Canvas Questions and GitHub contribution**
  - **Start early!**
- **Ut Prosim opportunities**
- **Next Class**
  - **Guest Lecture: Ben Nelson [Kinsale Insurance]**

AI Writing Code

Code Generation Models
Benchmarking
AI Usage in Practice

Learning Outcomes

By the end of the course, students should be able to:

- **Understand software engineering processes, methods, and tools used in the software development life cycle (SDLC)**
- **Use techniques and processes to create and analyze requirements for an application**
- **Use techniques and processes to design a software system**
- **Identify processes, methods, and tools related to phases of the SDLC**
- **Explain the differences between software engineering processes**
- **Discuss research questions and current topics related to software engineering**
- **Create and communicate about the requirements and design of a software application**

Warm-Up

Have you used AI tools for coding? Which tools? What was your experience?

1-2-4-All



🎂Person with closest birthday is “speaker” for group

Implementation



Goal: translate software design into a concrete system (i.e. write code)

- Can use any programming language, but some languages are better suited to certain types of programs than others
- SE/implementation is a *team* activity!
- *Software Artifacts*: source code, documentation, configuration files, media, executables, bug database, source code repository, issue trackers…

Code Generation with AI

“AI code-generation software is a class of tools that use artificial intelligence—usually large language models (LLMs)—to automatically produce computer code from natural language instructions or partial code snippets.” [Smalley]

Rather than having humans manually write all the code, AI-powered code generation tools can produce code automatically based on certain inputs and parameters.

- LLMs and generative AI tools are fundamentally changing the future of software engineering.

Future of SE

Is this the end of programmers?





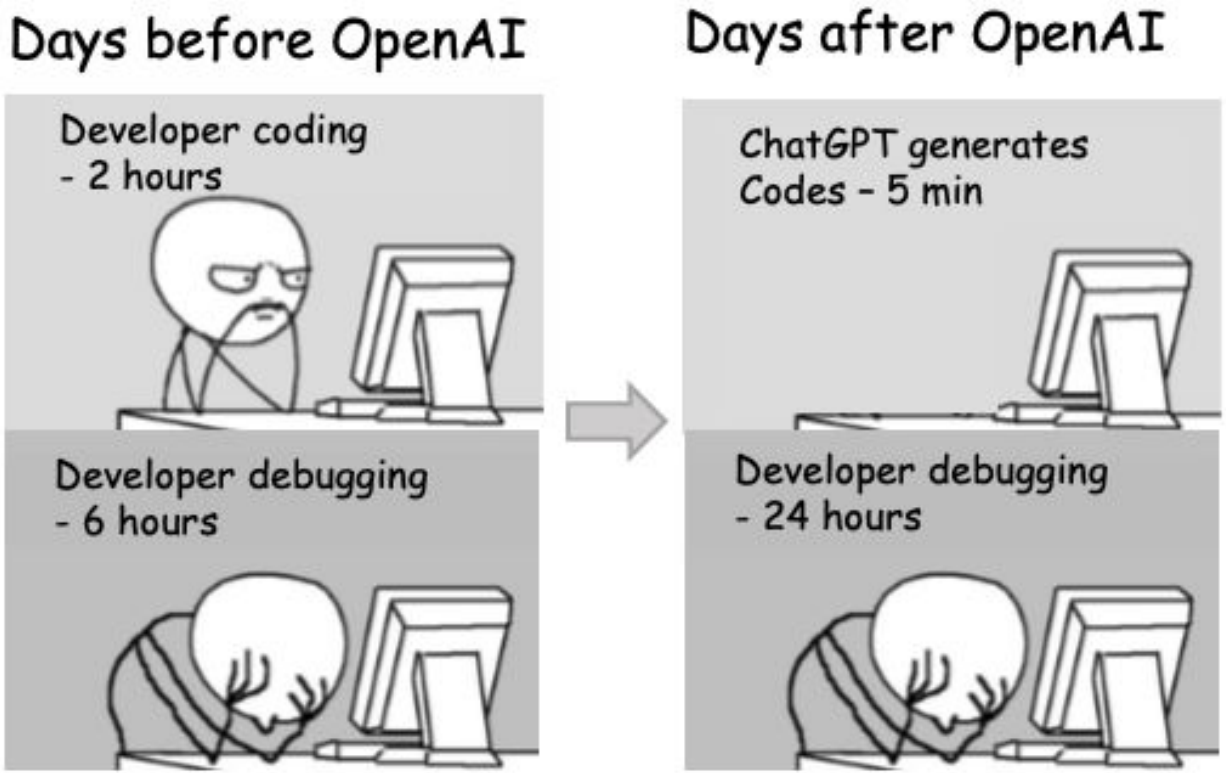




ChatGPT Will Replace Programmers Within 10 Years

Predicting The End of Manmade Software

Future of SE (cont.)



Brief History

AI for Code Generation in Pre-LLM era:

2016: Code summarization

2019: Code search, better summarization

- Code completion tools used smaller-scale version of current code generation processes!

2020: Beta version of GPT-3

2021: Initial Codex model and data

2023: GPT-4, GitHub Copilot

AROMA Code Recommendation with Extra Lines Highlighted

```
TextView licenseView = (TextView)
      findViewById(R.id.library_license_link);
SpannableString underlinedLicenseLink = new SpannableString(
   getString(R.string.library_license_link));
underlinedLicenseLink.setSpan(new UnderlineSpan(), 0,
      underlinedLicenseLink.length(), 0);
licenseView.setText(underlinedLicenseLink);
licenseView.setOnClickListener(v -> {
    FragmentManager fm = getSupportFragmentManager();
    LibraryLicenseDialog libraryLicenseDlg = new
          LibraryLicenseDialog();
    libraryLicenseDlg.show(fm, "fragment_license"); });
```

[Luan]

Motivations For and Against Usage

	Motivation	Distribution
	A. For using	
M1	To have an autocomplete or reduce the amount of keystrokes I make.	86% 6.2%
M2	To finish my programming tasks faster.	76% 12%
M3	To skip needing to go online to find specific code snippets, programming syntax, or API calls I'm aware of, but can't remember.	68% 14%
M4	To discover potential ways or starting points to write a solution to a problem I'm facing.	50% 24%
M5	To find an edge case for my code I haven't considered.	36% 44%
	B. For not using	
M6	Code generation tools write code that doesn't meet functional or non-functional (e.g., security, performance) requirements that I need.	54% 34%
M7	It's hard to control code generation tools to get code that I want.	48% 36%
M8	I spend too much time debugging or modifying code written by code generation tools.	38% 45%
M9	I don't think code generation tools provide helpful suggestions.	34% 46%
M10	I don't want to use a tool that has access to my code.	30% 51%
M11	I write and use proprietary code that code generation tools haven't seen before and don't generate.	28% 59%
M12	To prevent potential intellectual property infringement.	26% 66%
M13	I find the tool's suggestions too distracting.	26% 51%
M14	I don't understand the code written by code generation tools.	16% 76%
M15	I don't want to use open-source code.	10% 89%

Very important | Important | Moderately important | Slightly important | Not important at all

[Liang]

Large Language Models

AI code generation is possible due to recent advancements in machine learning, generative AI, and NLP:

- LLMs are trained on billions of input tokens
  - Natural language, code, etc.
  - Allows models to learn patterns
    - Reasoning capabilities, but not actual reasoning
- Models can interpret input prompts, then predict the next token to generate code.

How does AI code generation work?

- Step 1: Write a prompt

```
User: 🤦‍♂️ "Write a Python function to calculate the factorial of a number."
```

- Step 2: Tokenization

```
Input: "Write a Python function to calculate the factorial of a number."
Tokens: ["Write", "a", "Python", "function", "to", "calculate", "the", "factorial", "of",
"a", "number", "."]
```

- Step 3: Prediction Loop

```
LLM 🤓 (Hmm…Python functions usually start with “def”)
LLM: 🧠 "def"
LLM: 🧠 "def" + "factorial"
LLM: 🧠 "def factorial" + "("
LLM: 🧠 "def factorial(" + "n"
LLM: 🧠 "def factorial(n)" + ":"...
```

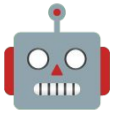
```
LLM: 🤖

def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n-1)
```

- Step 4: Attention/Refinement
- Step 5: Code Generation

[Gemini- “Explain how llm code generation works”]

AI Code Generation Approaches



LLM Code Generation: Conversational, predict tokens based on patterns in data
Agentic AI: Autonomous AI-powered agents break down complex tasks
Augmented Generation: Inject context from knowledge base into model reasoning
Finetuning: Aligning models to perform specific tasks based on exemplars

Vibe Coding



Andrej Karpathy 
@karpathy

There's a new kind of coding I call "vibe coding", where you fully give in to the vibes, embrace exponentials, and forget that the code even exists. It's possible because the LLMs (e.g. Cursor Composer w Sonnet) are getting too good. Also I just talk to Composer with SuperWhisper so I barely even touch the keyboard. I ask for the dumbest things like "decrease the padding on the sidebar by half" because I'm too lazy to find it. I "Accept All" always, I don't read the diffs anymore. When I get error messages I just copy paste them in with no comment, usually that fixes it. The code grows beyond my usual comprehension, I'd have to really read through it for a while. Sometimes the LLMs can't fix a bug so I just work around it or ask for random changes until it goes away. It's not too bad for throwaway weekend projects, but still quite amusing. I'm building a project or webapp, but it's not really coding - I just see stuff, say stuff, run stuff, and copy paste stuff, and it mostly works.

6:17 PM · Feb 2, 2025 · **5.1M** Views

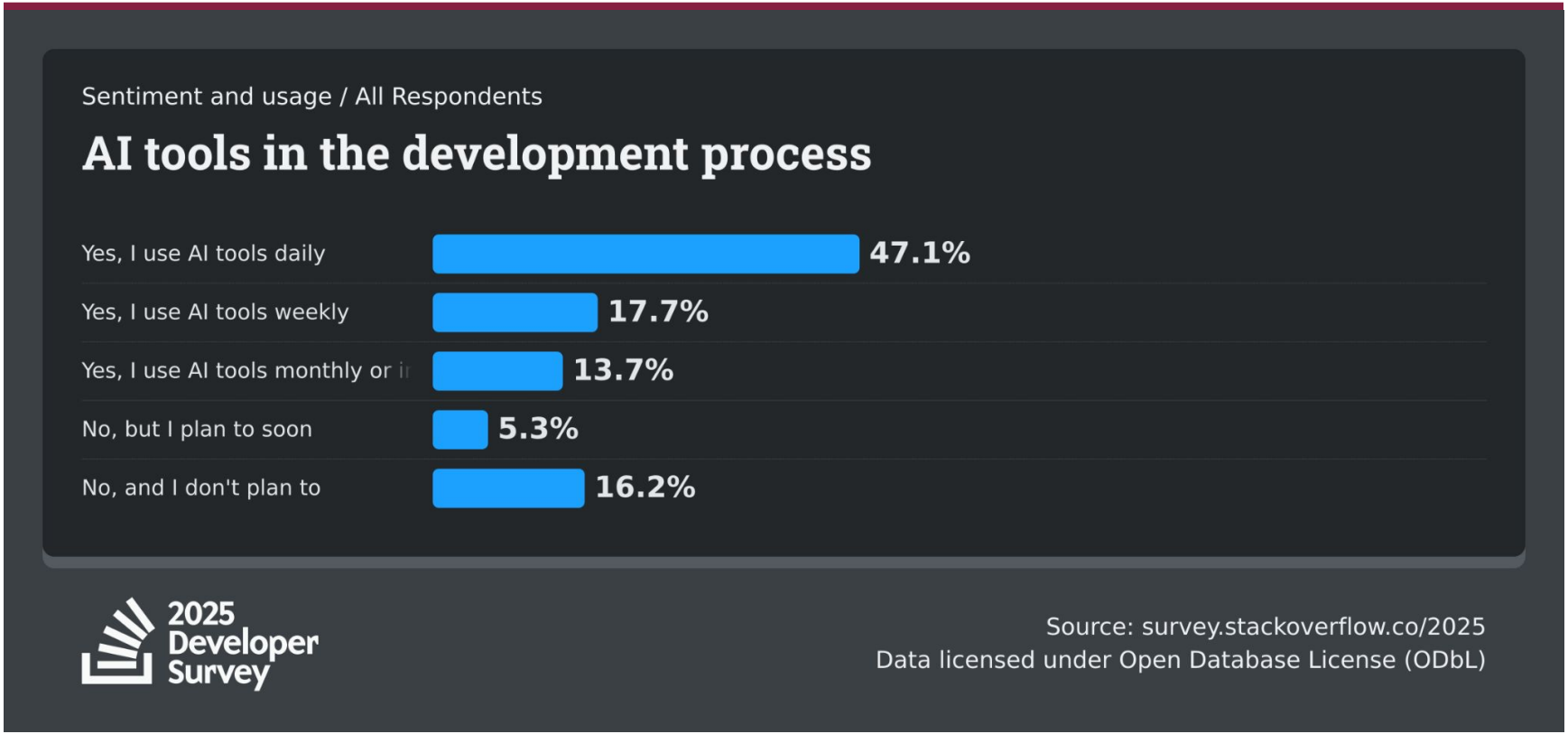
Based on what we've discussed in class so far, why is this a good or bad idea?

[Pandey]

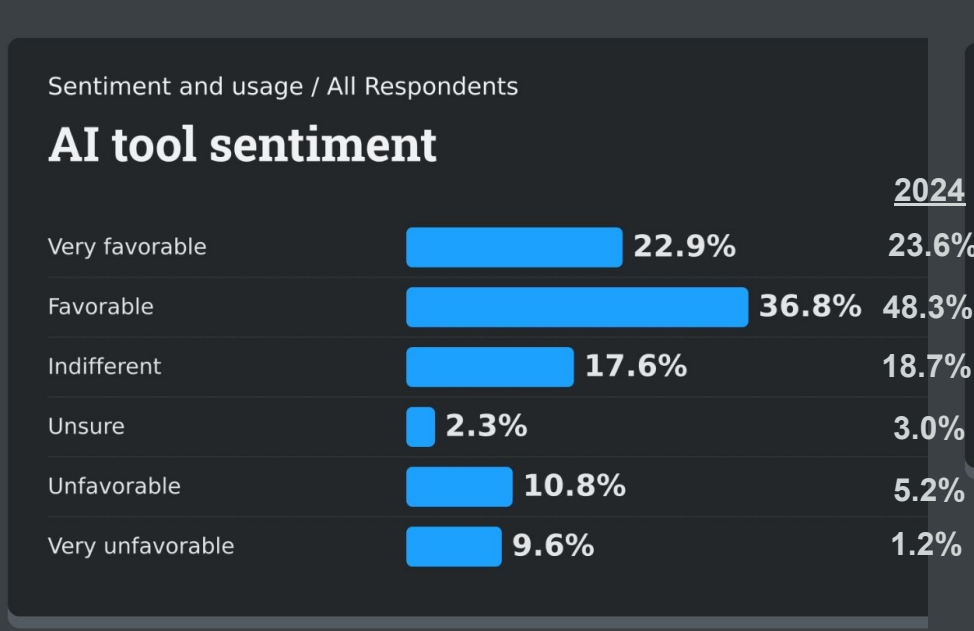
[Review]

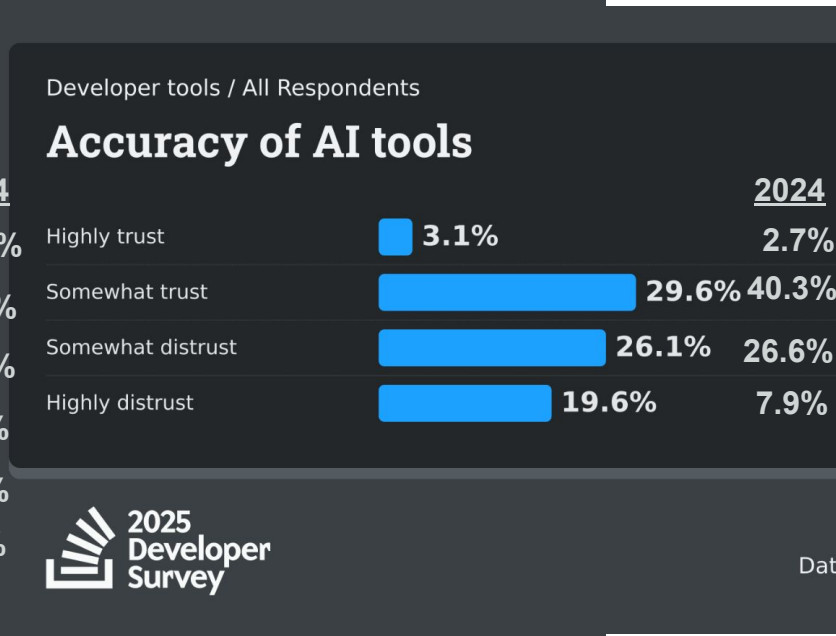
- No requirements from stakeholders
- No verification of requirements
- No design
- No usability testing
- More difficult to maintain
- Hard to debug
- More complex code
- …

Developer AI Sentiment and Usage



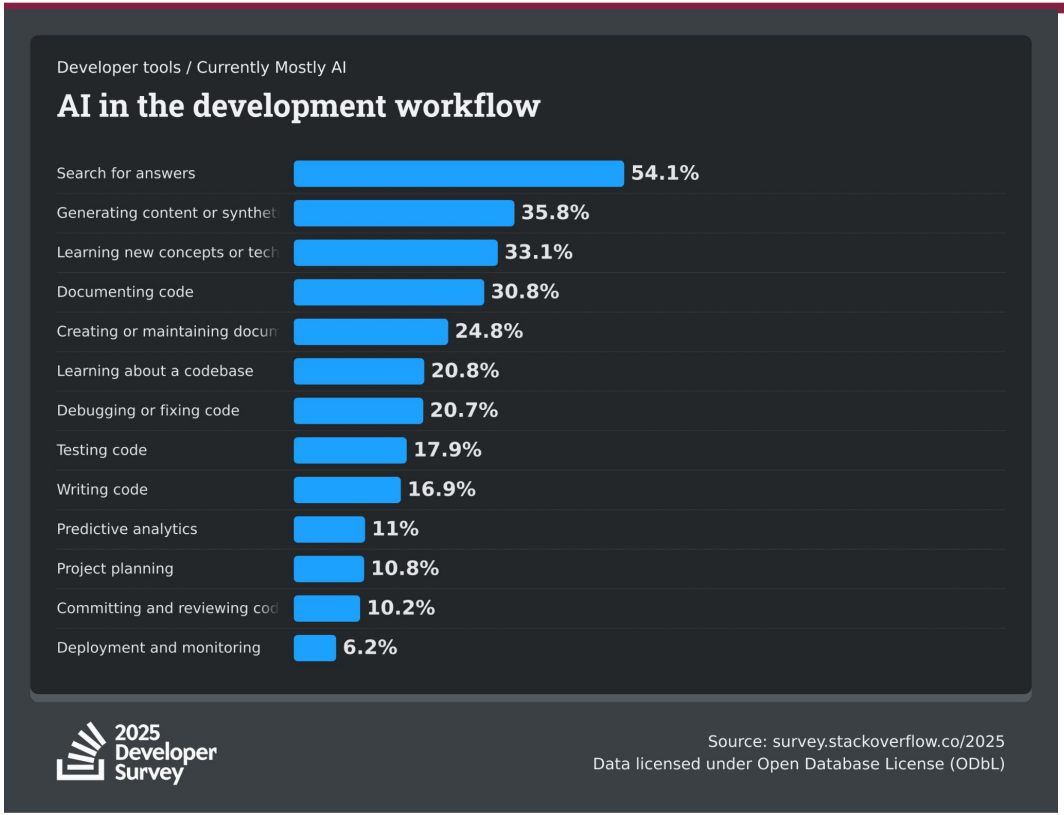
AI Sentiment and Usage (cont.)

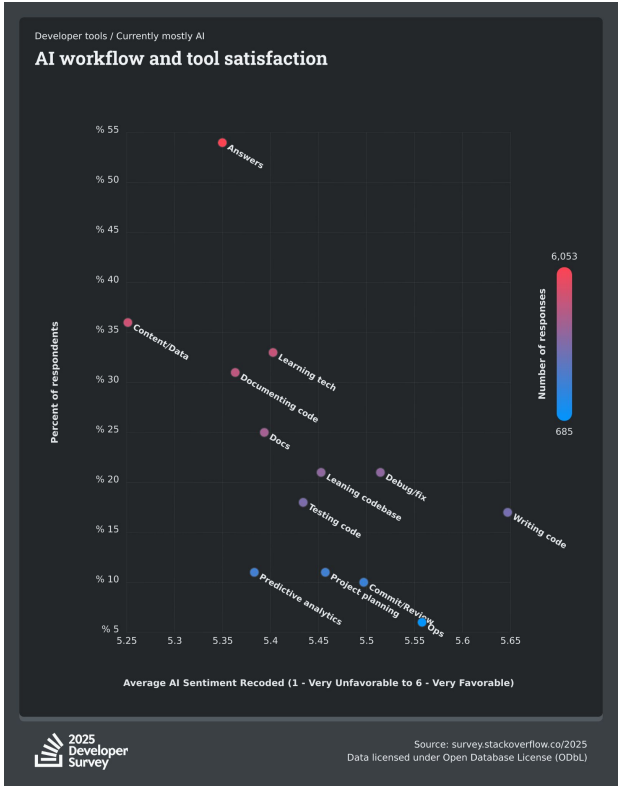




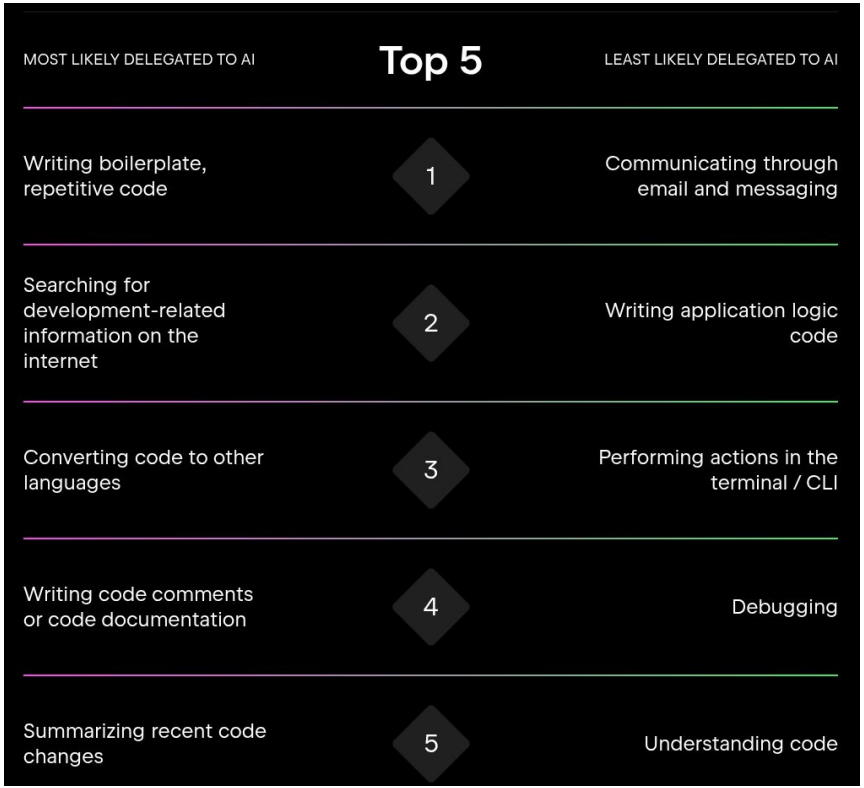
Decrease in sentiment/trust from last year!

Developer AI Workflow





Developer AI Workflow (cont.)



Humans vs. AI

ChatGPT generates more complex and longer programs than humans [Azeem]

Attribute	Source	Mean	Median	Std Dev	Range	IQR	Skewness	Kurtosis
Operators	ChatGPT	11.56	8.0	9.62	91	9	3.05	17.17
	Human	9.57	7.0	8.36	64	9	2.34	7.61
Distinct Operators	ChatGPT	3.73	3.0	2.13	11	3	0.99	0.98
	Human	3.32	3.0	2.04	11	2	1.06	1.12
Operands	ChatGPT	107.78	92.0	58.33	439	66	1.92	6.51
	Human	83.41	71.0	49.35	266	67	1.10	1.15
Distinct Operands	ChatGPT	47.97	46.0	16.10	127	20	1.69	7.09
	Human	38.45	36.0	16.74	101	23	0.79	1.05
Program Vocabulary	ChatGPT	51.71	49.0	16.92	133	21	1.71	7.46
	Human	41.76	40.0	17.17	110	23	0.81	1.29
Program Length	ChatGPT	119.34	101.5	65.18	527	72	2.11	8.32
	Human	92.98	84.0	53.75	292	72	1.07	1.06

Attribute	Source	Mean	Median	Std Dev	Range	IQR	Skewness	Kurtosis
Volume	ChatGPT	693.50	588.5	442.04	3900	460	2.68	13.63
	Human	515.48	445.5	345.67	2055	429	1.28	1.93
Difficulty	ChatGPT	4.45	3.0	3.74	33	4	2.95	15.38
	Human	3.71	3.0	3.13	19	3	2.09	5.64
Effort	ChatGPT	4113.71	2145.0	7136.76	73534	3629	5.60	42.71
	Human	2431.26	1216.0	3770.79	27035	2226	3.97	19.76
Cyclomatic Complexity	ChatGPT	6.02	5.0	3.56	24	3	1.55	3.33
	Human	4.95	4.0	2.94	16	4	1.23	1.51

Attribute	Source	Mean	Median	Std Dev	Range	IQR	Skewness	Kurtosis
Operators	ChatGPT	11.56	8.0	9.62	91	9	3.05	17.17
	Human	9.57	7.0	8.36	64	9	2.34	7.61
Distinct Operators	ChatGPT	3.73	3.0	2.13	11	3	0.99	0.98
	Human	3.32	3.0	2.04	11	2	1.06	1.12
Operands	ChatGPT	107.78	92.0	58.33	439	66	1.92	6.51
	Human	83.41	71.0	49.35	266	67	1.10	1.15
Distinct Operands	ChatGPT	47.97	46.0	16.10	127	20	1.69	7.09
	Human	38.45	36.0	16.74	101	23	0.79	1.05
Program Vocabulary	ChatGPT	51.71	49.0	16.92	133	21	1.71	7.46
	Human	41.76	40.0	17.17	110	23	0.81	1.29
Program Length	ChatGPT	119.34	101.5	65.18	527	72	2.11	8.32
	Human	92.98	84.0	53.75	292	72	1.07	1.06

Attribute	Source	Mean	Median	Std Dev	Range	IQR	Skewness	Kurtosis
Volume	ChatGPT	693.50	588.5	442.04	3900	460	2.68	13.63
	Human	515.48	445.5	345.67	2055	429	1.28	1.93
Difficulty	ChatGPT	4.45	3.0	3.74	33	4	2.95	15.38
	Human	3.71	3.0	3.13	19	3	2.09	5.64
Effort	ChatGPT	4113.71	2145.0	7136.76	73534	3629	5.60	42.71
	Human	2431.26	1216.0	3770.79	27035	2226	3.97	19.76
Cyclomatic Complexity	ChatGPT	6.02	5.0	3.56	24	3	1.55	3.33
	Human	4.95	4.0	2.94	16	4	1.23	1.51

Halstead's Metrics

- Metrics of the software should reflect the programming language.
- Calculations:

$\eta = \eta_1 \log_2 \eta_1 + \eta_2 \log_2 \eta_2$

Length: $N = N_1 + N_2$

Volume: $V = N * \log_2 \eta$

Difficulty: $D = \eta_1 / 2 * N_2 / \eta_2$

Effort: $E = D * V$

η_1 = the number of distinct operators

η_2 = the number of distinct operands

N_1 = the total number of operators

N_2 = the total number of operands

Potential Exam Question: Why is Halstead's metric different for the same program written in Java and Python?

```
main()
{
  int a, b, c, avg;
  scanf("%d %d %d", &a, &b, &c);
  avg = (a+b+c)/3;
  printf("avg = %d", avg);
}
```

The distinct *operators* (what action to perform) are:
`main, (), {}, int, scanf, &, =, +, /, printf, ,, ;` (12)

The distinct *operands* (what to apply actions to) are:
`a, b, c, avg, "%d %d %d", 3, "avg = %d"` (7)

Cyclomatic Complexity

The number of independent paths in a program.

- Also known as McCabe's Cyclomatic Complexity
- Measures the number of linear segments in a method (branches)
- Useful for measuring the *minimum* number of test cases for comprehe
- Calculate:
  - Number of decisions +

Definite Exam Question: Given the same code in [Java] and [Python], why is cyclomatic complexity the same and Halstead's metric different?

Open Challenges

- AI lacks project context 👨‍💻
- Current benchmarks are inaccurate 📊
  - For example, the most widely recognized benchmark (HumanEval) is:
    - *Mostly Python,*
    - *Leetcode-style programs,*
    - *Suffers from data leakage,*
    - *Limited unit tests,...*
- More complex debugging and understanding 🤔
  - Reviewing AI code increases cognitive effort [Tang]
- Security risks 🔒
  - 45% of AI generated code contains known vulnerabilities [Veracode]
- Ethical concerns (i.e., Licensing/Attribution) 📜

Class Activity

1. **Find a partner in class who was not in your Project 1 or Project 2 groups.**
2. **Select *three* LeetCode problems of similar difficulty**
3. **Implement as much as you can of the selected problems with your partner in three separate files, rotating between the driver and navigator (*active participant)*. Write code for each of the problems based on the following:**
   a. **For the first *n* minutes, only use the existing knowledge between you and your partner to solve the first problem.**
   b. **For the next *n* minutes, you can use static online resources (i.e., documentation and Stack Overflow) to solve the second problem.**
   c. **For the final *n* minutes, you can use AI tool of your choice (i.e., ChatGPT, GitHub Copilot, Claude Code etc.) to solve the third problem.***
4. **Submit the following on Canvas: [submit individually]**
   a. Three files correlating with the three selected problems from the activity.
   b. What are the advantages/disadvantages of using online resources and AI tools for coding?
   c. What are the advantages/disadvantages of pair programming?
   d. Reflect on the accuracy complexity of the code generated through each approach

* If you prefer to not use AI tools, repeat option a or b again.

Pair Programming





- The process of two software engineers working on a task at one computer.
  - One programmer, **the driver**, has control of the keyboard and mouse to create the implementation
  - The other programmer, **the navigator**, *actively* watches the driver's implementation to identify defects and participate in on-demand brainstorming

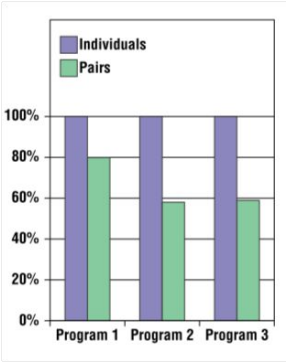
🔤 **Person whose name is first alphabetically will be the *driver* first.**

Table 1

Percentage of Test Cases Passed*

	Individuals	Pairs
Program 1	73.4	86.4
Program 2	78.1	88.6
Program 3	70.4	87.1
Program 4	78.1	94.4

*The difference in quality levels is statistically significant to $p < 0.01$; p is the probability that these results could occur by chance.



Class Activity

1. **Find a partner in class who was not in your Project 1 or Project 2 groups.**
2. **Select *three* LeetCode problems of similar difficulty**
3. **Implement as much as you can of the selected problems with your partner in three separate files, rotating between the driver and navigator (*active participant)*. Write code for each of the problems based on the following:**
   a. **For the first *n* minutes, only use the existing knowledge between you and your partner to solve the first problem.**
   b. **For the next *n* minutes, you can use static online resources (i.e., documentation and Stack Overflow) to solve the second problem.**
   c. **For the final *n* minutes, you can use AI tool of your choice (i.e., ChatGPT, GitHub Copilot, Claude Code etc.) to solve the third problem.***
4. **Submit the following on Canvas: [submit individually]**
   a. Three files correlating with the three selected problems from the activity.
   b. What are the advantages/disadvantages of using online resources and AI tools for coding?
   c. What are the advantages/disadvantages of pair programming?
   d. Reflect on the accuracy complexity of the code generated through each approach

* If you prefer to not use AI tools, repeat option a or b again.

References

- 025 genai code security report. Technical report, Veracode, 2025. https://www.veracode.com/wp-content/uploads/2025_GenAI_Code_Security_Report_Final.pdf.
- Josh Payne. *“Code Generation with LLMs”*. Stanford.
- Samina Azeem, Muhammad Shumail Naveed, Muhammad Sajid, Imran Ali. *“AI vs. Human Programmers: Complexity and Performance in Code Generation”*. Transactions on Computer Sciences, 2025.
- Jenny T. Liang, Chenyang Yang, Brad A. Myers. *“A Large-Scale Survey on the Usability of AI Programming Assistants: Successes and Challenges”*. International Conference on Software Engineering. 2024.
- Luan, Sifei, et al. *"Aroma: Code recommendation via structural code search".* Proceedings of the ACM on Programming Languages 3.OOPSLA (2019): 1-28.
- Brij Kishore Pandey. “LLM ≠ Generative AI ≠ AI Agents ≠ Agentic AI”. LinkedIn. 2025. https://www.linkedin.com/posts/brijpandeyji_llm-generative-ai-ai-agents-agentic-activity-7355965770012491776-jAH9/
- Ningzhi Tang, Meng Chen, Zheng Ning, Aakash Bansal, Yu Huang, Collin McMillan, Toby Jia-Jun Li. *“An Empirical Study of Developer Behaviors for Validating and Repairing AI-Generated Code”*. Workshop on the Intersection of HCI and PL, 2023.
- Adam Hughes. *“ChatGPT Will Replace Programmers Within 10 Years”*. LevelUp Coding, 2023.
- Stack Overflow Developer Survey. https://survey.stackoverflow.co/2025/
- Sarah Heckman